

KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Hafta 4 Laboratuvar Föyü

Kaynak Yönetimi ve Akıllı İşaretçiler

Doç. Dr. Tolga Berber · https://avesis.ktu.edu.tr/tberber

Tahmini süre: 2 saat
Gereksinimler: C++20 derleyici (g++/clang++), bir metin editörü

Öğrenme fazları: 1 • Ön-öğretim | 2 • İçerik | 3 • Katılım | 4 • Değerlendirme | 5 • İzleme

1 • Ön-öğretim — Neden Önemli?

Geçen hafta `Kitap` sınıfıyla kurucu ve yıkıcının biçimsel kurallarını öğrendiniz. Şimdi Hafta 2'de yazdığınız `araclar::IntTampon` yapısına, en masum görünen tek bir satırla dönün: `IntTampon b = a;`. O zaman `IntTampon`'un yalnızca bir kurucusu ve bir yıkıcısı vardı, kopyalanması hiç ele alınmamıştı — ve derleyici hiç uyarmadan iki nesneyi aynı belleğe bağlar, program bittiğinde aynı blok iki kez silinmeye çalışılır. Bu föyde bu sessiz tuzağı önce göreceksiniz, sonra derin kopya ve akıllı işaretçilerle adım adım kendiniz kapatacaksınız.

1 • Ön-öğretim — Ön Bilgi

Bu föy YG I ders notu Bölüm 4 (Kaynak Yönetimi ve Akıllı İşaretçiler) üzerine kuruludur ve şu kavramları hazır varsayar: Hafta 2'den `araclar::IntTampon` (ham `int* veri`, `int boyut`; `new[]`/`delete[]` ile elle bellek yönetimi), Hafta 3'ten kurucu/yıkıcının biçimsel terimleri (üye başlatma listesi, `this`), sınıf ve kapsülleme. Kopya kurucu, kopya atama, Rule of 3 ve akıllı işaretçi kavramlarını bu föyde elinizle kuracağınız için önceden bilmeniz gerekmez; ilgili biçimsel tanımlar ders notu Bölüm 4'ün dört ünitesinde işlenir.

Bu föyde derleyicinin ürettiği varsayılan kopyanın neden tehlikeli olduğunu kavramsal olarak göreceksiniz; sonra kendi kopya kurucunuzu ve kopya atama operatörünüzü yazıp *derin kopya* ile bağımsız nesneler elde edeceksiniz (Rule of 3). Ardından aynı sorumluluğu elle taşımak yerine kütüphaneye devrettiğiniz `unique_ptr` ve `shared_ptr` akıllı işaretçileriyle bir `AkilliTampon` sınıfı yazacaksınız. Föyü, düğümlerini `unique_ptr` ile yöneten ilk bağlı veri yapınız `IntBagliListe` ile kapatacaksınız.

Bu föyü şöyle kullanın: her adımda önce *Ne yapacağız?* kutusundaki görevi tamamlayın (kodu yazın, ardından verilen kabuk komutunu çalıştırın), sonra kendi sonucunuzu *Beklenen* kutusu ve çıktı örneğiyle karşılaştırarak doğru yolda olduğunuzu hemen doğrulayın. Bir şey tutmazsa, sonraki adıma geçmeden o adımı düzeltin — her adım bir öncekinin üstüne inşa edilir. Aradaki *Kontrol Noktası*, o ana kadar yaptıklarınızı bir arada sınamanıza yarar. Bölümler ders notu Bölüm 4'ün üniteleriyle ilerler; her bölümün başında ilgili üniteye bir atıf bulacak, takıldığınızda oraya dönebileceksiniz.

1 Kopyalama Sorunu: Sığ Kopya Neden Tehlikeli?

(*bkz. Ders notu Bölüm 4, Ünite 1 — Kopyalama Sorunu: Sığ Kopya Neden Tehlikeli?*)

Adım 1: Proje iskeletini kurup `IntTampon`'u geri getirin

Ne yapacağız? `hafta04-projesi` adlı bir klasör oluşturacak, içine `int_tampon.hpp` ve `int_tampon.cpp` dosyalarını Hafta 2'deki hâliyle (yalnızca alanlar, kurucu ve yıkıcı bildirimi/ tanımı; kopyalamayla ilgili hiçbir şey yok) yeniden yazacaksınız.

Neden? Bu haftanın anlatı ekseni Hafta 2'de doğan bu yapıyı yeniden ele almaktır. O zaman `IntTampon` yalnızca `new[]` ile bellek ayırıp `delete[]` ile bırakıyordu; kopyalama davranışı derleyicinin insafına bırakılmıştı. Bu adımda önce o eksik hâli aynen geri getiriyorsunuz — bir sonraki adımda bu eksikliğin neden tehlikeli olduğunu göreceksiniz.

```
8     int* veri;
9     int boyut;

12    IntTampon(int n);
13    ~IntTampon();
```

```
8  IntTampon::IntTampon(int n) {
9     veri = new int[n];
10    boyut = n;
11    std::cout << "tampon olusturuldu (boyut=" << boyut << ")\n";
12 }

16 IntTampon::~IntTampon() {
17    delete[] veri;
18    std::cout << "tampon temizlendi\n";
19 }
```

`araclar` ad alanını ve `#pragma once` korumasını unutmayın — ikisi de Hafta 2'deki gibi kalır.

```
>_ Terminal

$ mkdir hafta04-projesi && cd hafta04-projesi
$ g++ -std=c++20 -c int_tampon.cpp -o int_tampon.o
```

Beklenen sonuç: Komut hatasız/uyarısız tamamlanır ve dizinde bir `int_tampon.o` nesne dosyası oluşur. `IntTampon` bu noktada Hafta 2'deki hâliyle birebir aynıdır: yalnızca kurucusu ve yıkıcısı vardır, kopya kurucu ve kopya atama operatörü henüz yazılmamıştır.

Adım 2: Sığ kopyanın tehlikesini kavramsal olarak inceleyin

Ne yapacağız? Kod yazmadan önce, eğer `IntTampon`'a hiçbir kopya kurucu/atama eklemeseydiniz derleyicinin ne üreteceğini ve bunun neden tehlikeli olacağını çıkaracaksınız.

Neden? Siz hiçbir şey yazmazsanız derleyici, sınıfınız için otomatik olarak bir **varsayılan kopya kurucu** ve bir **varsayılan kopya atama operatörü** üretir. Bunlar **üye üye** (memberwise) çalışır: her alanı olduğu gibi kopyalar. `int boyut` gibi bir alan için bu sorunsuzdur, ama `int* veri` gibi bir işaretçi alanı için bu, işaretçinin kendisini (yani içindeki adresi) kopyalamak demektir — işaret ettiği veriyi **değil**. Buna **sığ kopya** denir. Sonuç olarak `araclar::IntTampon x = a;` yazsaydınız, `x.veri` ile `a.veri` **aynı** heap adresini gösterirdi: biri veriyi değiştirse öteki de değişmiş görünürdü (istenmeyen paylaşılan durum) ve ikisi de kapsam dışına çıktığında aynı bellek bloğu **iki kez** `delete[]` edilmeye çalışılırdı — çifte `delete`, tanımsız davranış. Bu adımda bu senaryoyu **gerçekten çalıştırmayacaksınız** (programınızı bilerek bozmayacaksınız); yalnızca deseni tanıyacaksınız.

Adım 2 (devamı)

Analoji

Sığ kopya, aynı eve iki ayrı zarfla aynı adresi göndermeye benzer: iki zarfta da (iki nesnede de) yazan adres birebir aynıdır, ikisi de aynı eve (aynı bellek bloğuna) açılır. Zarflardan biri "evi yık" derse öbür zarfın sahibi de evsiz kalır; ikisi de "evi yık" derse aynı ev iki kez yıkılmaya çalışılır — tıpkı çifte `delete` gibi. Derin kopya ise evin birebir kopyasını yeni bir arsada inşa etmeye benzer: artık iki bağımsız ev vardır, biri yıkılsa öteki hiç etkilenmez. Bir sonraki bölümde `IntTampon`'u tam olarak bu ikinci duruma taşıyacaksınız.

Dikkat

Bu senaryo yalnızca bir düşünce deneyidir. `araclar::IntTampon x = a;` satırını gerçekten yazıp iki nesneyi de kapsam dışına çıkarmayın — gerçek bir çifte `delete` çalıştırırsanız programınız çöker (tanımsız davranış). Amaç riski görmek, riski tetiklemek değildir.

Beklenen sonuç: Bir sınıf ham bir işaretçi/kaynak tutuyorsa, derleyicinin ürettiği varsayılan kopyanın neden tehlikeli olduğunu (sığ kopya → paylaşılan adres → çifte `delete` veya istenmeyen paylaşılan durum) kendi cümlelerinizle açıklayabiliyor olmanız gerekir.

2 Derin Kopya: Kopya Kurucu ve Kopya Atama

(bkz. Ders notu Bölüm 4, Ünite 1 — Kopyalama Sorunu: Sığ Kopya Neden Tehlikeli?, "Çözüm: kendi kopyanızı elle yazmak".)

Adım 3: Kopya kurucu ve kopya atama operatörünü bildirin ve tanımlayın

Ne yapacağız? `int_tampon.hpp` dosyasına kopya kurucu (`IntTampon(const IntTampon&)`) ve kopya atama operatörü (`IntTampon& operator=(const IntTampon&)`) bildirimlerini, `int_tampon.cpp` dosyasına da bunların *derin kopya* yapan gövdelerini ekleyeceksiniz.

Neden? Bir önceki adımda gördüğünüz sığ kopya riskini ortadan kaldırmanın yolu, bu iki özel üye fonksiyonu **kendiniz** yazmaktır. İkisi de aynı deseni izler: önce kendi `new[]`'ini yaparak bağımsız bir bellek bloğu ayırır, sonra elemanları tek tek kopyalar (döngüyle) — bu yüzden *derin kopya* denir. Kopya atama operatöründe iki ek adım kritiktir: (1) **öz-atama kontrolü** (`if (this == &diger)`) — bir nesneyi kendisine atarsanız (`a = a;`), önce eski belleği silip sonra ondan okumaya çalışmak felaket olurdu; (2) atamadan önce **eski belleği bırakmak** (`delete[] veri;`) — aksi hâlde nesnenin önceki kaynağı sızıntıya dönüşür.

```
16    IntTampon(const IntTampon& diger);
17    IntTampon& operator=(const IntTampon& diger);
```

```
23 IntTampon::IntTampon(const IntTampon& diger) {
24    boyut = diger.boyut;
25    veri = new int[boyut];
26    for (int i = 0; i < boyut; ++i) {
27       veri[i] = diger.veri[i];
28    }
29    std::cout << "tampon kopyalandi (derin kopya, boyut=" << boyut << ")\n";
30 }
```

Adım 3 (devamı)

```
IntTampon& IntTampon::operator=(const IntTampon& diger) {
   if (this == &diger) {
      return *this; // oz-atama kontrolu
   }
   delete[] veri; // eski kaynak birakilir
   boyut = diger.boyut;
   veri = new int[boyut];
   for (int i = 0; i < boyut; ++i) {
      veri[i] = diger.veri[i];
   }
   std::cout << "tampon atandi (derin kopya, boyut=" << boyut << ")\n";
   return *this;
}
```

Kopya atama operatörünün `if (this == &diger)` satırını atlamayın: bu, `diger`'in adresiyle `this` işaretçisinin (mevcut nesnenin kendi adresi) aynı olup olmadığını kontrol eder.

```
>_ Terminal

$ g++ -std=c++20 -c int_tampon.cpp -o int_tampon.o
```

Beklenen sonuç: Komut hatasız tamamlanır. `IntTampon` artık dört özel üye fonksiyona sahiptir: kurucu, yıkıcı, kopya kurucu, kopya atama operatörü — henüz hiçbirini çalıştırıp gözlemlemediniz, bunu bir sonraki adımda yapacaksınız.

Adım 4: Derin kopyanın bağımsızlığını demoyla gözlemleyin

Ne yapacağız? `int_tampon_kopya_demo.cpp` dosyasında bir `IntTampon a` oluşturup `IntTampon b = a;` ile kopyalayacak, ardından yalnızca `a`'yı değiştirip `b`'nin etkilenmediğini yazdıracaksınız; aynısını bir kopya atama (`c = a;`) ile de tekrarlayacaksınız.

Neden? Bu adım, bir önceki adımda yazdığınız derin kopyanın gerçekten işe yaradığının kanıtıdır: `IntTampon b = a;` çağrıldığında artık sığ kopya değil, sizin yazdığınız kopya kurucu çalışır — `b`, `a`'dan bağımsız kendi `new[]`'ini alır. Bu yüzden `a.veri[0]`'ı değiştirmek `b.veri[0]`'ı hiç etkilemez. Aynı bağımsızlık, var olan bir nesneye atama yaptığınızda (`c = a;`) da geçerlidir — bu sefer kopya atama operatörünüz çalışır.

```
   std::cout << "-- kopya kurucu demo --\n";
   araclar::IntTampon a(3);
   for (int i = 0; i < a.boyut; ++i) {
      a.veri[i] = i + 1; // a = {1,2,3}
   }
   araclar::IntTampon b = a; // kopya kurucu cagrilir (derin kopya)
   a.veri[0] = 99; // yalnizca a degisir
   std::cout << "a.veri[0]=" << a.veri[0] << " b.veri[0]=" << b.veri[0] << "\n";
```

```
   std::cout << "-- kopya atama demo --\n";
   araclar::IntTampon c(3);
   c = a; // kopya atama operatoru cagrilir (derin kopya)
   a.veri[1] = 42; // yalnizca a tekrar degisir, c etkilenmez
   std::cout << "a.veri[1]=" << a.veri[1] << " c.veri[1]=" << c.veri[1] << "\n";
```

Adım 4 (devamı)

Terminal

```
$ g++ -std=c++20 int_tampon_kopya_demo.cpp int_tampon.cpp -o
↪  int_tampon_kopya_demo
$ ./int_tampon_kopya_demo
```

Beklenen sonuç: Konsolda kurucu/kopya kurucu/kopya atama mesajları ile birlikte iki karşılaştırma satırı görürsünüz: ilkinde `a.veri[0]` 99 iken `b.veri[0]` hâlâ 1'dir (b etkilenmedi); ikincisinde `a.veri[1]` 42 iken `c.veri[1]` hâlâ 2'dir (c etkilenmedi). Bu, iki nesnenin artık tamamen bağımsız birer kaynağa sahip olduğunun kanıtıdır.

Beklenen Çıktı

```
-- kopya kurucu demo --
tampon olusturuldu (boyut=3)
tampon kopyalandi (derin kopya, boyut=3)
a.veri[0]=99 b.veri[0]=1
-- kopya atama demo --
tampon olusturuldu (boyut=3)
tampon atandi (derin kopya, boyut=3)
a.veri[1]=42 c.veri[1]=2
tampon temizlendi
tampon temizlendi
tampon temizlendi
```

Son üç `tampon temizlendi` satırına dikkat edin: bunlar `main` sona ererken `c`, `b`, `a` nesnelerinin (kuruluşlarının tersi sırayla) yıkıcılarının otomatik çalışmasıdır — her biri kendi bağımsız belleğini bırakır, çifte `delete` riski yoktur.

Kontrol Noktası — devam etmeden doğrulayın

`int_tampon.hpp` dosyanızda dört özel üye fonksiyon bildirimi (kurucu, yıkıcı, kopya kurucu, kopya atama) olduğundan ve `./int_tampon_kopya_demo` çıktınızda `b.veri[0]`'ın 1, `c.veri[1]`'in 2 kaldığından emin olun — bu iki değer değişseydi, kopya kurucu/atamanız hâlâ sığ kopya yapıyor demektir.

Adım 5: Sığ kopya olsaydı ne olurdu ve taşımaya kısa bir bakış

Ne yapacağız? Aynı `int_tampon_kopya_demo.cpp` dosyasına, (1) kullanıcı tanımlı kopya kurucu/atama olmasaydı ne olacağını özetleyen yalnızca bir yorum bloğu ve (2) `std::move` ile kısa bir taşıma önizlemesi (`IntTampon d(std::move(c));`) ekleyeceksiniz.

Neden? Birinci ekleme, Ünite 1'de kavramsal olarak gördüğünüz sığ kopya riskini şimdi elinizde çalışan bir derin kopya sınıfınız varken geriye dönük olarak özetler — artık karşılaştıracak bir "doğru" hâliniz var. İkinci ekleme ise *taşıma* (*move*) fikrine yalnızca kısa bir bakıştır: `std::move(c)`, `c`'yi bir "rvalue referansı" olarak işaretler ve normalde bir kaynağı kopyalamadan devretmeyi *amaçlar*. Ama `IntTampon` için henüz kullanıcı tanımlı bir taşıma kurucusu **yazmadınız** — bu yüzden `std::move(c)` yine de derlenir, ama derleyici uygun bir taşıma kurucusu bulamadığı için **kopya kurucuya düşer**: güvenlidir, ama gereksiz bir derin kopya daha yapılmış olur (verimsiz). Bu haftada bir taşıma kurucusu elle yazmayacaksınız; gerçek kazancı Ünite 3'te `unique_ptr`'ın taşımayı *zorunlu kılmasıyla* göreceksiniz.

```
27    // Not: IntTampon icin kullanici tanimli kopya kurucu/atama olmasaydi,
```

Adım 5 (devamı)

```
    // derleyici varsayilan (sig/uye-uye) kopyayi uretirdi. O durumda
    // `araclar::IntTampon x = a;` ifadesinde x.veri ve a.veri AYNI heap
    // adresini gosterirdi (yalnizca isaretci/adres kopyalanir, isaret
    // edilen veri kopyalanmaz). Ikisi de kapsam disina ciktiginda ayni
    // bellek blogu iki kez `delete[]` edilirdi (cifte delete, tanimsiz
    // davranis). Bu kesit yalnizca desen/aciklama icindir; gercek bir
    // tanimsiz davranis burada TETIKLENMEZ.
```

```
    std::cout << "-- tasima onizleme --\n";
    araclar::IntTampon d(std::move(c));
    // Not: IntTampon icin kullanici tanimli bir tasima kurucusu YAZILMADI;
    // std::move(c) yine de derlenir ama burada -- tasima kurucusu
    // olmadigi icin -- kopya kurucuya duser (guvenli, fakat gereksiz bir
    // derin kopya yapilir; verimsiz). c bu yuzden hala gecerli kalir.
    std::cout << "d.veri[1]=" << d.veri[1] << "\n";
```

Terminal

```
$ g++ -std=c++20 int_tampon_kopya_demo.cpp int_tampon.cpp -o
↪  int_tampon_kopya_demo
$ ./int_tampon_kopya_demo
```

Beklenen sonuç: Önceki adımın çıktısına, sonuna eklenen taşıma önizlemesi yüzünden üç yeni satır eklenir: bir ”tasima onizleme” başlığı, bir ”tampon kopyalandi” mesajı (çünkü `std::move(c)` yine de kopya kurucuya düştü) ve `d.veri[1]=2`. Yorum bloğu ekrana hiçbir şey basmaz, yalnızca kaynak kodda okunur bir açıklama olarak durur. Son dört `tampon temizlendi` satırı artık `d`, `c`, `b`, `a` için sırayla çalışır.

Beklenen Çıktı

```
-- kopya kurucu demo --
tampon olusturuldu (boyut=3)
tampon kopyalandi (derin kopya, boyut=3)
a.veri[0]=99 b.veri[0]=1
-- kopya atama demo --
tampon olusturuldu (boyut=3)
tampon atandi (derin kopya, boyut=3)
a.veri[1]=42 c.veri[1]=2
-- tasima onizleme --
tampon kopyalandi (derin kopya, boyut=3)
d.veri[1]=2
tampon temizlendi
tampon temizlendi
tampon temizlendi
tampon temizlendi
```

3 Rule of 3: `=default` ve `=delete`

(*bkz. Ders notu Bölüm 4, Ünite 2 — Rule of 3, `=default`/`=delete` ve Taşımaya Kısa Bir Bakış.*)

Adım 6: Rule of 3'ü tamamlayın: `Sayac` ve `TekilKart`

Ne yapacağız? `rule_of_3_ornekleri.hpp` dosyasında iki küçük karşıt sınıf yazacaksınız: yalnızca bir `int` tutan `Sayac` (kopya kurucu/atama `= default;`) ve kopyalanmaması gereken bir `TekilKart` (kopya kurucu/atama `= delete;`); ardından `rule_of_3_demo.cpp` içinde ikisini de deneyeceksiniz.

Neden? Buraya kadar gördükleriniz **Rule of 3** ilkesinin somut bir uygulamasıydı: bir sınıf yıkıcı, kopya kurucu veya kopya atama operatöründen **birini** elle yazmak zorundaysa (çünkü bir kaynağın sahibidir), genelde **üçünü de** yazmak zorundadır — üçü de aynı soruyu ("bu sınıf bir kaynağın sahibi mi?") cevaplar. `Sayac` farklı bir durumu gösterir: yalnızca bir `int` tuttuğu için derleyicinin ürettiği memberwise kopya zaten doğrudur — bunu `= default;` ile **açıkça** ifade edersiniz ("varsayılanı kullan" demek, hiçbir şey yazmamaktan farklıdır: niyetinizi okunur kılar). `TekilKart` ise tam tersi bir durumu gösterir: kopyalanmaması **gereken** bir kaynağı temsil eder; kopya kurucu ve atamayı `= delete;` ile tamamen yasaklarsınız — derleyici, biri bu sınıfı kopyalamaya çalıştığı anda derleme hatası verir. Bu, Ünite 3'teki `unique_ptr`'ın kopyalanamaz oluşunun habercisidir.

```
class Sayac {
public:
    explicit Sayac(int baslangic = 0) : deger_(baslangic) {}
    Sayac(const Sayac&) = default;
    Sayac& operator=(const Sayac&) = default;
    int deger() const { return deger_; }
    void artir() { ++deger_; }

private:
    int deger_;
};
```

```
class TekilKart {
public:
    explicit TekilKart(int numara) : numara_(numara) {}
    TekilKart(const TekilKart&) = delete;
    TekilKart& operator=(const TekilKart&) = delete;
    int numara() const { return numara_; }

private:
    int numara_;
};
```

```
    std::cout << "-- Sayac (=default) --\n";
    araclar::Sayac s1(5);
    araclar::Sayac s2 = s1; // kopya kurucu = default, sorunsuz calisir
    s2.artir();
    std::cout << "s1.deger()=" << s1.deger() << " s2.deger()=" << s2.deger() << "\n";

    std::cout << "-- TekilKart (=delete) --\n";
    araclar::TekilKart k1(42);
    std::cout << "k1.numara()=" << k1.numara() << "\n";
    // Asagidaki satir KASITLI OLARAK yorumda birakilmistir; acilirsa
    // derleme hatasi verir (kopya kurucu = delete edildigi icin):
    // araclar::TekilKart k2 = k1;
```

Adım 6 (devamı)

Analoji

Rule of 3, bir set menü/kombo sipariş etmeye benzer: bir kaynağı özel olarak yönetmeye karar verdiğinizde (garson yerine kendiniz servis yapıyorsunuz demektir), menünün üç parçasını (yıkıcı, kopya kurucu, kopya atama) birlikte sipariş etmeniz gerekir — yalnız birini alıp diğerlerini "standart" bırakmak menüyü yarım bırakır ve tutarsızlığa (sızıntı veya çifte `delete`) yol açar. `=default` ve `=delete` ise bir başvuru formundaki iki kutuya benzer: "bu alan için standart cevabı kullan" kutusu (`=default`) ile "bu seçenek kapalıdır, işaretlenemez" damgası (`=delete`) — derleyiciye hangisini istediğinizi açıkça söylersiniz.

Terminal

```
$ g++ -std=c++20 rule_of_3_demo.cpp -o rule_of_3_demo
$ ./rule_of_3_demo
```

Beklenen sonuç: `Sayac` sorunsuz kopyalanır: `s1.deger()` 5 kalırken `s2.deger()`, `artir()` çağrısından sonra 6 olur. `TekilKart` bölümünde yalnızca `k1.numara()` yazdırılır; kaynak koddaki yorum satırındaki `araclar::TekilKart k2 = k1;` satırını siz de açıp derlemeyi denerseniz (yalnız denemek için, sonra tekrar yorum satırına çevirin), kopya kurucu `=delete` edildiği için bir derleme hatası alırsınız.

Beklenen Çıktı

```
-- Sayac (=default) --
s1.deger()=5 s2.deger()=6
-- TekilKart (=delete) --
k1.numara()=42
```

Ortak Hata / Tuzak

`TekilKart k2 = k1;` satırını gerçekten açıp derlemeyi denediğinizde `use of deleted function` gibi bir hata görüp paniklemeyin — bu hatanın **amacı budur**: derleyici, sizi kopyalanmaması gereken bir kaynağı yanlışlıkla kopyalamaktan koruyor. Denemeyi bitirince satırı yeniden yorum satırına çevirmeyi unutmayın, yoksa dosyanın geri kalanı derlenmez.

4 Akıllı İşaretçiler: `unique_ptr` ve `shared_ptr`

(bkz. Ders notu Bölüm 4, Ünite 3 — Akıllı İşaretçiler: `unique_ptr` ve `shared_ptr`.)

Adım 7: `AkilliTampon` sınıfını `unique_ptr` ile yazın (Rule of 0)

Ne yapacağız? `akilli_tampon.hpp` dosyasında, tek üyesi `std::unique_ptr<int[]>` olan bir `AkilliTampon` sınıfı bildirecek, `akilli_tampon.cpp` dosyasında da kurucusunu `std::make_unique<int[]>(n)` ile tanımlayacaksınız.

Neden? Buraya kadar `IntTampon`'u Rule of 3 ile **elle** doğru hale getirdiniz — bu hâlâ hataya açıktır: üç fonksiyonun her birini doğru yazmak sorumluluğu size aitti. Akıllı işaretçiler bu sorumluluğu **kütüphaneye** devreder. `unique_ptr<T>`, bir kaynağı **tekil ve**

Adım 7 (devamı)

kopyalanamaz biçimde sahiplenen, yalnızca `std::move` ile **taşınabilen** bir akıllı işaretçidir; `std::make_unique<T>(...)` ile oluşturulur, çıplak `new` hiç yazılmaz. `AkilliTampon`'a dikkat edin: hiçbir kullanıcı tanımlı yıkıcı, kopya kurucu veya kopya atama operatörü **yazılmadı** — bu, **Rule of 0**'ın somut bir örneğidir: kaynak yönetimini zaten üstlenen bir üye (`unique_ptr`) kullandığınızda, derleyicinin ürettiği memberwise davranış otomatik olarak doğru hâle gelir (ve `unique_ptr` üyesi zaten kopyalanamaz olduğu için sınıfın kendisi de **kendiliğinden** kopyalanamaz hale gelir — Ünite 2'nin `=delete` fikri burada örtük olarak gerçekleşir).

```
class AkilliTampon {
public:
   explicit AkilliTampon(int n);
   int boyut() const { return boyut_; }
   int& operator[](int i) { return veri_[i]; }
   int operator[](int i) const { return veri_[i]; }

private:
   std::unique_ptr<int[]> veri_;
   int boyut_;
};
```

```
AkilliTampon::AkilliTampon(int n)
   : veri_(std::make_unique<int[]>(n)), boyut_(n) {
   std::cout << "akilli tampon olusturuldu (boyut=" << boyut_ << ")\n";
}
```

```
>_ Terminal

$ g++ -std=c++20 -c akilli_tampon.cpp -o akilli_tampon.o
```

Beklenen sonuç: Komut hatasız tamamlanır. `AkilliTampon` dosyanızda `new`/`delete` anahtar kelimelerinden hiçbiri geçmez — tüm bellek yönetimi `std::unique_ptr<int[]>` ve `std::make_unique` üzerinden yapılır.

Adım 8: `unique_ptr` ile taşımayı deneyin

Ne yapacağız? `akilli_isaretciler_demo.cpp` dosyasında bir `AkilliTampon a(5)` oluşturup dolduracak, `AkilliTampon b = std::move(a);` ile sahipliği `b`'ye taşıyacak ve `b`'nin içeriğini yazdıracaksınız.

Neden? Bu adımda Ünite 2'nin taşıma önizlemesinden farklı olarak `std::move` artık gerçek bir sahiplik devri anlamına gelir: `AkilliTampon` içindeki `unique_ptr<int[]>` üyesi kopyalanamadığı için `AkilliTampon b = a;` (kopya) hiç **derlenmez** — yalnızca `AkilliTampon b = std::move(a);` (taşıma) derlenir ve bellek sahipliğini `a`'dan `b`'ye devreder; `a` bu noktadan sonra artık kendi belleğine sahip değildir. Kaynak dosyada bu gerçeği kanıtlayan bir yorum satırı da bulacaksınız: `AkilliTampon c = b;` satırı kasıtlı olarak yorumda bırakılmıştır, çünkü açılırsa derleme hatası verir.

```
   std::cout << "-- unique_ptr demo --\n";
   araclar::AkilliTampon a(5);
   for (int i = 0; i < a.boyut(); ++i) {
      a[i] = i * 10;
   }
   araclar::AkilliTampon b = std::move(a); // sahiplik b'ye tasinir
   std::cout << "b: ";
```

Adım 8 (devamı)

```
    for (int i = 0; i < b.boyut(); ++i) {
        std::cout << b[i] << " ";
    }
    std::cout << "\n";
    // Asagidaki satir DERLENMEZ: unique_ptr uyesi kopyalanamaz oldugu
    // icin AkilliTampon de kendiliginden kopyalanamaz hale gelir:
    // araclar::AkilliTampon c = b;
```

Analoji

`unique_ptr`, devredilebilir ama çoğaltılamaz tek bir bagaj fişine benzer: fiş elden ele verilebilir (taşınabilir, `std::move`) ama fotokopisi çekilemez (kopyalanamaz); fiş kimdeyse bagajın (kaynağın) sahibi odur — aynı anda iki kişide aynı fiş olamaz.

Terminal

```
$ g++ -std=c++20 akilli_isaretciler_demo.cpp akilli_tampon.cpp \
    int_tampon.cpp -o akilli_isaretciler_demo
$ ./akilli_isaretciler_demo
```

Beklenen sonuç: Konsolda kurucu mesajının ardından `b`'nin beş elemanının (`0 10 20 30 40`) yazdırıldığını görürsünüz — taşımadan sonra `b` tam olarak `a`'nın doldurduğu değerlere sahiptir, hiçbir yeni bellek ayrılmamış, yalnızca sahiplik devredilmiştir.

Beklenen Çıktı

```
-- unique_ptr demo --
akilli tampon olusturuldu (boyut=5)
b: 0 10 20 30 40
```

Adım 9: `shared_ptr` ile referans sayımını (`use_count`) gözlemleyin

Ne yapacağız? Aynı `akilli_isaretciler_demo.cpp` dosyasına, `std::make_shared<araclar::IntTampon>(5)` ile bir `p1` oluşturacak, iç içe bir blokta `p2 = p1;` atayacak ve `use_count()`'un blok öncesinde/içinde/sonrasında nasıl değiştiğini yazdıracaksınız.

Neden? `shared_ptr<T>`, `unique_ptr`'ın tersine **paylaşılan sahiplik** sağlar: birden çok `shared_ptr` aynı kaynağı aynı anda işaret edebilir. Dahili bir **referans sayacı** tutulur (`use_count()` ile okunur); her yeni kopya sayacı artırır, her kapsam dışına çıkış sayacı azaltır, **son** sahip kapsam dışına çıktığında kaynak otomatik bırakılır. `p2`, `p1`'in kapsandığı iç bloğun sonunda yok olduğunda sayaç 2'den 1'e düşer ama kaynağın kendisi silinmez — çünkü `p1` hâlâ yaşıyor. Bunu Ünite 1'in `IntTampon b = a;` örneğiyle karşılaştırın: orada `a` ve `b` birer `IntTampon` nesnesiydi ve derin kopyayla **bağımsız** iki kaynak elde etmiştiniz; burada `p1`/`p2` birer `shared_ptr<IntTampon>`'dur ve **aynı** nesneyi paylaşırlar — iki farklı niyet, iki farklı araç. `std::make_shared<T>(...)`, `make_unique` gibi çıplak `new` yazmadan oluşturur; ayrıca tek bir bellek ayırmayla hem nesneyi hem referans sayacını birlikte kurduğu için ayrı bir `new` + `shared_ptr` kurmaktan daha verimlidir.

```
    std::cout << "-- shared_ptr demo --\n";
    auto p1 = std::make_shared<araclar::IntTampon>(5);
    std::cout << "p1.use_count()=" << p1.use_count() << "\n";
```

Adım 9 (devamı)

```
30    {
31        auto p2 = p1; // paylasilan sahiplik, sayac artar
32        std::cout << "p1.use_count()=" << p1.use_count()
33                  << " p2.use_count()=" << p2.use_count() << "\n";
34    } // p2 kapsam disina cikar, sayac azalir
35    std::cout << "p1.use_count()=" << p1.use_count() << "\n";
```

Analoji

`shared_ptr`, ortak kullanılan bir aile aboneliğine benzer: kaç kişi aboneliği (kaynağı) kullanıyorsa sayaç o kadardır; abonelikten ayrılanlar sayıldıkça sayaç düşer, **son** kişi de ayrılınca abonelik (kaynak) tamamen kapanır.

Terminal

```
$ g++ -std=c++20 akilli_isaretciler_demo.cpp akilli_tampon.cpp \
    int_tampon.cpp -o akilli_isaretciler_demo
$ ./akilli_isaretciler_demo
```

Beklenen sonuç: `unique_ptr` demosunun altında, `use_count()` değerinin sırayla 1, sonra (iç blok içinde) 2, blok kapandıktan sonra tekrar 1 olarak yazdırıldığını görürsünüz — referans sayacının paylaşan/sahip sayısını gerçek zamanlı izlediğinin kanıtıdır.

Beklenen Çıktı

```
-- unique_ptr demo --
akilli tampon olusturuldu (boyut=5)
b: 0 10 20 30 40
-- shared_ptr demo --
tampon olusturuldu (boyut=5)
p1.use_count()=1
p1.use_count()=2 p2.use_count()=2
p1.use_count()=1
tampon temizlendi
```

İpucu

Varsayılan tercihiniz her zaman `unique_ptr` olsun — en ucuz ve en açık sahiplik biçimidir. `shared_ptr`'ı yalnızca gerçekten birden çok sahip **aynı anda** kaynağı yaşatmak zorundaysa seçin; "belki paylaşılır" ihtimaline karşı önden `shared_ptr` kullanmak gereksiz referans sayma maliyeti getirir.

Kontrol Noktası — devam etmeden doğrulayın

`akilli_isaretciler_demo` çıktınızda `use_count()` sırasının $1 \to 2 \to 1$ olduğundan ve `AkilliTampon c = b;` satırının kaynak dosyada hâlâ yorumda kaldığından (açarsanız derlenmeyeceğini unutmayın) emin olun.

5 Uygulama: `IntBagliListe` (Kütüphanenin İlk Bağlı Yapısı)

(bkz. Ders notu Bölüm 4, Ünite 4 — Uygulama: `IntBagliListe` — Kütüphanenin İlk Bağlı Yapısı.)

Adım 10: `Dugum` tipini ve arayüzü bildirin

Ne yapacağız? `int_bagli_liste.hpp` dosyasında, kendi türünden bir sonrakini `unique_ptr` ile sahiplenen iç içe bir `Dugum` tipi, listenin tek üyesi `bas_` ve `IntBagliListe` sınıfının public arayüzünü (`ekleBasa`, `yazdir`, `bosMu`) bildireceksiniz.

Neden? `IntBagliListe`, tek yönlü ve somut tipli (yalnız `int` tutan) bir bağlı liste olacak — şablonlar (generic) Hafta 9'un konusu, bu hafta girmiyorsunuz. `Dugum` tipinin `std::unique_ptr<Dugum> sonraki` alanı, tekil sahipliğin bağlı listede ne kadar doğal olduğunu gösterir: zincirdeki her düğümü yalnızca **bir** "önceki" sahiplenir (liste başıysa listenin kendisi, değilse önceki düğüm) — tıpkı `unique_ptr`'ın izin verdiği gibi. Listenin tek üyesi `std::unique_ptr<Dugum> bas_` (baş düğüm, liste boşsa `nullptr`) olduğu için, yıkıcıyı elle yazmanıza gerek **yoktur** (yine Rule of 0): `bas_` yok edilirken zincirdeki her `sonraki` de otomatik ve ardışık olarak yok edilir.

```
18    struct Dugum {
19       int deger;
20       std::unique_ptr<Dugum> sonraki;
21    };

25    std::unique_ptr<Dugum> bas_;

10    IntBagliListe() = default;
11    void ekleBasa(int deger);
12    void yazdir() const;
13    bool bosMu() const { return bas_ == nullptr; }
```

Analoji

Bağlı liste/düğüm zinciri, define avında art arda gelen ipucu kağıtlarına benzer: her ipucu bir sonraki ipucunun yerini gösterir; son ipucunda "define burada, başka ipucu yok" yazar (`nullptr`). Her ipucu yalnız **bir** sonraki ipucunu "sahiplenir" — kimse aynı ipucuna iki yerden aynı anda sahip değildir; tıpkı `unique_ptr` ile birebir uyum içinde.

```
>_ Terminal

$ g++ -std=c++20 -fsyntax-only int_bagli_liste.hpp
```

Beklenen sonuç: Komut hatasız tamamlanır — bu, header dosyasının tek başına sözdizimsel olarak geçerli olduğunu (henüz tanım/gövde içermediği için bağlanabilir bir program üretmez) doğrular.

Adım 11: `ekleBasa` ve `yazdir` gövdelerini tanımlayıp listeyi deneyin

Ne yapacağız? `int_bagli_liste.cpp` dosyasında `ekleBasa` ve `yazdir` üye fonksiyonlarını tanımlayacak, ardından `int_bagli_liste_demo.cpp` içinde üç değer ekleyip listeyi yazdıracaksınız.

Neden? `ekleBasa(int deger)`, `make_unique<Dugum>` ile yeni bir düğüm kurar, onun `sonraki`'sine **mevcut** `bas_`'ı `std::move` ile aktarır, sonra yeni düğümü `bas_`'a yine `std::move` ile atar — sahiplik zinciri hiç kırılmadan kaydırılır (bu üç satırın sırası önemlidir: önce eski `bas_` yeni düğümün `sonraki`'sine taşınmalı, sonra yeni düğüm `bas_`'a taşınmalıdır). `yazdir() const` ise zinciri gezerken **sahiplik almayan** bir **gözlemci ham işaretçi** (`const Dugum*`, `.get()` ile alınır) kullanır — önemli bir ayrım: gezinme/okuma sahiplik gerektir-

Adım 11 (devamı)

mez, yalnızca `unique_ptr`'ın **sahip olduğu** ilişkiler akıllı işaretçiyle temsil edilir. `unique_ptr` zaten taşınamaz/ kopyalanamaz kullanılırsa, gezinme için de bir `unique_ptr` kullanmaya çalışmak zincirin sahiplik yapısını bozardı.

```
8  void IntBagliListe::ekleBasa(int deger) {
9     auto yeniDugum = std::make_unique<Dugum>();
10    yeniDugum->deger = deger;
11    yeniDugum->sonraki = std::move(bas_); // eski bas, yeni dugumun sonrakisi olur
12    bas_ = std::move(yeniDugum); // sahiplik yeni dugume kayar
13 }
```

```
17 void IntBagliListe::yazdir() const {
18    const Dugum* gozlemci = bas_.get(); // sahiplik almayan gozlemci isaretci
19    while (gozlemci != nullptr) {
20       std::cout << gozlemci->deger << " -> ";
21       gozlemci = gozlemci->sonraki.get();
22    }
23    std::cout << "nullptr\n";
24 }
```

```
6     araclar::IntBagliListe liste;
7     liste.ekleBasa(3);
8     liste.ekleBasa(2);
9     liste.ekleBasa(1);
10    liste.yazdir();
```

Analoji

Gözlemci ham işaretçi, bir kitapçıda rafları gezen bir müşteriye benzer: kitapları görüp okuyabilir, ama hiçbirinin sahibi değildir; dükkândan çıkarken elinde hiçbir şey götürmez (kaynağı serbest bırakma sorumluluğu ona ait değildir) — yalnızca `.get()` ile "bakar", sahiplik almaz.

Terminal

```
$ g++ -std=c++20 int_bagli_liste_demo.cpp int_bagli_liste.cpp \
    -o int_bagli_liste_demo
$ ./int_bagli_liste_demo
```

Beklenen sonuç: `ekleBasa` her çağrıldığında yeni değeri listenin **başına** ekler (LIFO sıra); sırasıyla 3, 2, 1 eklediğiniz için `yazdir()` listeyi 1'den başlayarak, sondan `nullptr` ile biten tek satırlık bir zincir olarak yazdırır.

Beklenen Çıktı

```
1 -> 2 -> 3 -> nullptr
```

Ortak Hata / Tuzak

`ekleBasa` içinde satır sırasını tersine çevirip önce `bas_ = std::move(yeniDugum);` sonra `yeniDugum->sonraki = std::move(bas_);` yazmak yaygın bir hatadır: bu sırada `bas_` zaten yeni düğüme taşınmış olur, ikinci satır artık **taşınmış** (**boşaltılmış**) bir `bas_`'ı `sonraki`'ye taşımaya çalışır — sonuç, yalnızca tek düğümlük, bir sonraki adıma hiç bağlanmayan bozuk bir liste olur.

☑ Kontrol Noktası — devam etmeden doğrulayın

`int_bagli_liste_demo` çıktınızın tam olarak `1 -> 2 -> 3 -> nullptr` olduğundan emin olun; sıra tersse (`3 -> 2 -> 1`) `ekleBasa` yerine listenin **sonuna** ekleyen bir mantık yazmış olabilirsiniz.

Adım 12: `CMakeLists.txt` oluşturup tüm hedefleri tek seferde derleyin

? Ne yapacağız? Şu ana kadar dört ayrı programı (`int_tampon_kopya_demo`, `rule_of_3_demo`, `akilli_isaretciler_demo`, `int_bagli_liste_demo`) her seferinde elle `g++` komutu yazarak derlediniz. Şimdi tek bir `CMakeLists.txt` dosyasıyla dördünü de tek komutla derleyeceksiniz.

ℹ Neden? Hafta 2'de gördüğünüz gibi, proje dosya sayısı arttıkça elle `g++` komutlarını yönetmek zorlaşır; bu haftaki dört ayrı hedef için de aynı iskeleti kullanacaksınız. Bu adım aynı zamanda tüm haftanın çalışmasını tek bir tutarlı derleme adımında toplamanızı sağlar.

```
cmake_minimum_required(VERSION 3.20)
project(hafta04_temel CXX)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
add_executable(int_tampon_kopya_demo int_tampon_kopya_demo.cpp int_tampon.cpp)
add_executable(rule_of_3_demo rule_of_3_demo.cpp)
add_executable(akilli_isaretciler_demo akilli_isaretciler_demo.cpp akilli_tampon.cpp
    int_tampon.cpp)
add_executable(int_bagli_liste_demo int_bagli_liste_demo.cpp int_bagli_liste.cpp)
```

`akilli_isaretciler_demo` hedefinin üç dosyadan (`akilli_isaretciler_demo.cpp` + `akilli_tampon.cpp` + `int_tampon.cpp`) oluştuğuna dikkat edin — bu hedef hem `AkilliTampon`'u hem de `make_shared` ile sarılan `IntTampon`'u kullanıyordu.

Terminal

```
$ cmake -B build
$ cmake --build build
$ ./build/int_tampon_kopya_demo
$ ./build/rule_of_3_demo
$ ./build/akilli_isaretciler_demo
$ ./build/int_bagli_liste_demo
```

👁 Beklenen sonuç: İlk komut `build/` klasörünü oluşturup derleme sistemini hazırlar, ikinci komut dört hedefi de tek seferde derler; sonraki dört komut her programı ayrı ayrı çalıştırır ve bu föyde daha önce gördüğünüz çıktıların birebir aynısını üretir.

Beklenen Çıktı

```
[100%] Built target int_tampon_kopya_demo
[100%] Built target rule_of_3_demo
[100%] Built target akilli_isaretciler_demo
[100%] Built target int_bagli_liste_demo
```

Görevinizi tamamladıktan sonra, aşağıdakilerden **birini** seçip kendiniz ekleyin:

- `IntBagliListe`'ye, listenin toplam düğüm sayısını dönen bir `int uzunluk() const` üye fonksiyonu ekleyin (gözlemci ham işaretçiyle zinciri gezip sayarak); `int_bagli_liste_demo.cpp` içinde üç eleman ekledikten sonra çağırıp sonucu yazdırın.

Adım 12 (devamı)

- `AkilliTampon`'a, `std::make_shared<araclar::IntTampon>` demosundaki karşılaştırma fikrinden yola çıkarak, `operator[]` ile sınır dışı bir indekse (ör. `boyut()`'tan büyük bir `i`) erişilmeye çalışıldığında ekrana bir hata mesajı yazıp programı `return` ile güvenle sonlandıran (segfault yerine) bir kontrol ekleyin.

İpucu

Bu haftanın en sık karşılaşılan hataları: (1) kopya atama operatöründe `if (this == &diger)` öz-atama kontrolünü unutmak — `a = a;` gibi bir çağrıda eski bellek silinip sonra ondan okunmaya çalışılır; (2) kopya kurucu/atamayı yazarken `new[]` boyutunu `diger.boyut` yerine yanlışlıkla `boyut` (henüz atanmamış) ile ayırmak; (3) `AkilliTampon b = a;` (kopya) ile `AkilliTampon b = std::move(a);` (taşıma) sözdizimini karıştırıp "use of deleted function" hatası almak; (4) `shared_ptr` demosunda `p2`'yi `p1` ile aynı kapsamda bırakıp `use_count()`'un hiç düşmediğini görmek (iç bloğun kapanması gerekir); (5) `ekleBasa` içinde `std::move` satırlarının sırasını ters yazıp bozuk bir liste oluşturmak (bkz. yukarıdaki Ortak Hata kutusu).

Sorun Giderme

Belirti	Olası neden	Çözüm
`use of deleted function` (`TekilKart` için)	Kopya kurucusu `=delete` edilmiş bir sınıf kopyalanmaya çalışıldı	Bu davranış kasıtlıdır; kopyalamak yerine bir referans (`TekilKart&`) kullanın ya da nesneyi taşıyın
`double free or corruption` / program çöküyor	Kopya kurucu/ atama yazılmadan (ya da eksik yazılmış) iki nesne aynı `veri` adresini paylaşıyor	`IntTampon`'un kopya kurucu ve kopya atama operatörünün ikisinin de `new[]` ile kendi belleğini ayırdığından emin olun
`b.veri[0]` de `a` ile birlikte değişiyor	Kopya kurucu hâlâ sığ kopya yapıyor (yalnızca işaretçiyi kopyalıyor, döngüyle elemanları kopyalamıyor)	`kopya_kurucu_govde` kesitindeki `for` döngüsünü eksiksiz kopyaladığınızı kontrol edin
`AkilliTampon c = b;` satırı derlenmiyor	Bu satır kasıtlı olarak yorumda bırakılmıştı; açıldıysa `unique_ptr` üyesi kopyalanamadığı için sınıf da kopyalanamaz	Satırı tekrar yorum satırına çevirin; kopya yerine `std::move` kullanın
`use_count()` beklenenden yüksek kalıyor	`p2`, iç blok kapanmadan önce ölçüldü ya da iç blok hiç kapatılmadı	Ölçümü `{ }` bloğunun kapanışından **sonra** yaptığınızdan emin olun
Liste çıktısı `3 -> 2 -> 1 -> nullptr` çıkıyor (beklenenin tersi)	`ekleBasa` çağrı sırası ya da `std::move` satırlarının sırası yanlış	Üç çağrının `ekleBasa(3)`, `ekleBasa(2)`, `ekleBasa(1)` sırasıyla yapıldığını ve kaynak kesitteki satır sırasını birebir kopyaladığınızı doğrulayın

5 · İzleme — Aktarım — Başka Nerede?

Bugün elinizle kurduğunuz beceriler yalnızca `IntTampon`'a ya da `IntBagliListe`'ye özgü değildir; kaynak tutan her tipte tekrar tekrar karşınıza çıkar:

- **Kitaplık projesine:** "Bir kaynak tutan sınıf kopyalanırken derin kopya gerekir" kararı, ileride ham bir kaynak (bellek, dosya tanıtıcısı) tutacak her tip için (bir `Uye` kaydının ham tamponu, bir `Odunc` defterinin dinamik dizisi) yeniden verilecek: bu nesneyi kopyalarken kaynağı paylaşayım mı, yeni bir kaynak mı ayırayım? Tek fiziksel kopyası olması gereken bir kaydı (aynı anda iki rafta görünmemesi gereken bir `Odunc`) modellerken de kopyalamayı buradaki `= delete` ile yasaklarsınız.
- **Akıllı işaretçilere:** Bu haftadan sonra varsayılan tercihiniz her zaman `unique_ptr` olsun — ham `new`/`delete` yazdığınız her yerde onu düşünün; yalnızca birden çok raf/kaydın **aynı** `Kitap` nesnesini aynı anda yaşatması gerektiğinde `shared_ptr`'a geçin. Bir bağlı listenin düğümlerini bugün `unique_ptr` ile bağladınız ve tek bir `delete` bile yazmadınız; aynı Rule of 0 rahatlığını Kitaplık'a eklenecek her sahiplik ilişkisinde arayın.
- **Bağlı yapılara ve gözlemci işaretçiye:** "Sahiplik ve gezinme ayrı sorumluluklardır" ilkesi, ağaç ve graf gibi her bağlı yapıda karşınıza çıkacak: zinciri/yapıyı `unique_ptr` sahiplenir, üstünde `.get()` ile alınan sahiplenmeyen bir gözlemci işaretçiyle gezersiniz.
- **İleriye (Hafta 5):** Bu hafta kaynaklarınızı güvenle kopyaladınız ve akıllı işaretçilerle sardınız; gelecek hafta bu nesnelerin `[]`, `<<`, karşılaştırma gibi dilin yerleşik operatörleriyle doğal biçimde kullanılmasını (Operatör Yükleme) göreceksiniz. `IntBagliListe`'nin `yazdir()`'i de o haftadan itibaren bir `operator<<` olarak ifade edilebilir hâle gelecektir.